

目录

md_to_html.py 使用说明

一、直接复制使用的命令

二、使用前需要准备什么

三、最常用的命令

1. 把一个 Markdown 转成 HTML
2. 同时生成 HTML 和 PDF
3. 生成带目录的 HTML 和 PDF
4. 转换整个文件夹里的md文档
5. 不在命令中填写路径

四、推荐的命令

五、参数通俗解释

`--pdf` : 生成 PDF

`--toc` : 生成目录

`--strict` : 严格检查 Markdown

`--skip-existing` : 跳过已有文件

`-o` 或 `--output` : 指定输出位置

`--number-sections` : 自动给标题编号

`--browser` : 手动指定浏览器

`--lang` : 设置 HTML 语言

六、图片应该怎么写

七、怎么看懂运行结果

八、常见问题

提示 “未找到 Pandoc”

提示 “未找到 Chrome 或 Edge”

HTML 正常，但目录出现了一整段正文

中文是否会乱码

PDF 没有更新

路径中有中文或空格怎么办

九、参数速查表

md_to_html.py 使用说明

`md_to_html.py` 用来把 Markdown 文档转换成适合阅读和打印的 HTML，也可以继续调用 Chrome 或 Edge，自动生成 PDF。

简单理解，它完成的是下面这条流程：

```
Markdown 文件 → HTML 网页 → PDF 文件
```

HTML 会保留下来，方便先在浏览器中检查排版。PDF 使用浏览器的打印引擎生成，可以正常处理中文字体、表格和本地图片，不需要逐个点击“打印”和“另存为 PDF”。

一、直接复制使用的命令

只生成 HTML：

```
python md_to_html.py "Markdown编写规范.md"
```

生成 HTML 和 PDF，不要目录：

```
python md_to_html.py "Markdown编写规范.md" --pdf
```

生成 HTML、目录和 PDF：

```
python md_to_html.py "." --toc --pdf
```

正式交付前严格检查并生成完整结果：

```
python md_to_html.py "." --toc --pdf --strict
```

只补充尚未生成的文件：

```
python md_to_html.py "." --toc --pdf --strict --skip-existing
```

完整的 Markdown 写法要求请查看同目录下的《Markdown 编写规范》。

二、使用前需要准备什么

运行脚本需要：

1. Python 3;
2. Pandoc;
3. 如果需要生成 PDF，还需要 Chrome 或 Edge。

本机已经安装这些程序时，一般不需要额外配置。脚本会自动寻找 Pandoc、Chrome 和 Edge。

打开 PowerShell，然后进入脚本所在目录：

```
cd "md2pdf"
```

检查脚本是否可以运行：

```
python md_to_html.py --help
```

如果能看到参数说明，表示 Python 和脚本运行正常。

三、最常用的命令

1. 把一个 Markdown 转成 HTML

```
python md_to_html.py "Markdown编写规范.md"
```

运行后，会在 Markdown 同目录下生成：

```
Markdown编写规范.html
```

2. 同时生成 HTML 和 PDF

```
python md_to_html.py "Markdown编写规范.md" --pdf
```

运行后，会生成：

Markdown编写规范.html

Markdown编写规范.pdf

3. 生成带目录的 HTML 和 PDF

```
python md_to_html.py "Markdown编写规范.md" --toc --pdf
```

只有写了 `--toc` 才会生成目录。不需要目录时，直接省略这个参数。

4. 转换整个文件夹里的md文档

脚本位于 `md2pdf` 中时，用 `.` 表示当前文件夹：

```
python md_to_html.py "." --toc --pdf
```

脚本会查找该文件夹及其所有子文件夹中的 `.md` 和 `.markdown` 文件，然后逐个转换。

5. 不在命令中填写路径

也可以直接运行：

```
python md_to_html.py
```

看到提示后，把文件夹路径粘贴进去，再按 Enter：

请输入包含 Markdown 文档的文件夹地址。

文件夹地址：.

路径外面有英文双引号也没关系，脚本会自动去掉。

四、推荐的命令

正式批量转换建议启用格式检查、目录和 PDF：

```
python md_to_html.py "." --toc --pdf --strict
```

参数含义：

- `--toc`：生成目录；
- `--pdf`：生成 PDF；
- `--strict`：发现高风险 Markdown 格式问题时，停止转换有问题的文件。

推荐操作顺序：

1. 按照《Markdown 编写规范》完成文档；
2. 使用上面的严格模式命令转换；
3. 打开 HTML，检查目录、标题、表格和图片；
4. 打开 PDF，检查分页、字体和图片清晰度；
5. 确认无误后再交付。

五、参数通俗解释

`--pdf`：生成 PDF

不写 `--pdf` 时，脚本只生成 HTML：

```
python md_to_html.py "Markdown编写规范.md"
```

写上 `--pdf` 后，同时生成 HTML 和 PDF：

```
python md_to_html.py "Markdown编写规范.md" --pdf
```

PDF 与 HTML 放在同一输出目录，文件名相同，扩展名不同。

`--toc`：生成目录

需要目录时：

```
python md_to_html.py "Markdown编写规范.md" --toc
```

不需要目录时，不写 `--toc`。目录默认收集一至三级标题，因此正文标题应规范使用 `#`、`##` 和 `###`。

`--strict`：严格检查 Markdown

脚本每次都会检查常见格式问题。普通模式只显示警告，然后继续转换。

加入 `--strict` 后，如果某个 Markdown 存在以下问题，该文件会停止转换：

- 标题的 `#` 后缺少空格；
- 分隔线或 Setext 标记前后缺少空行；
- 代码块没有结束标记；
- 本地图片不存在。

用法：

```
python md_to_html.py "." --strict
```

严格模式只能检查明确的格式问题。目录层级是否合理、表格内容是否正确、PDF 分页是否美观，仍然需要打开 HTML 和 PDF 查看。

`--skip-existing`：跳过已有文件

如果文件已经生成，不想重复转换，可以使用：

```
python md_to_html.py "." --pdf --skip-existing
```

HTML 和 PDF 会分别判断：

- HTML 已存在、PDF 不存在：跳过 HTML，只补 PDF；
- PDF 已存在、HTML 不存在：生成 HTML，跳过 PDF；
- 两者都存在：两者都跳过。

不写 `--skip-existing` 时，脚本会重新生成输出文件。新的 PDF 生成并验证成功后，才会替换原来的 PDF，避免浏览器失败时破坏已有文件。

`-o` 或 `--output`：指定输出位置

默认情况下，HTML 和 PDF 会放在 Markdown 旁边。

把单个文件输出到指定目录：

```
python md_to_html.py "Markdown编写规范.md" --output ".\output" --pdf
```

转换整个文件夹并统一输出到另一个目录：

```
python md_to_html.py "." --output ".\output" --toc --pdf
```

批量转换时，脚本会在输出目录中保留原来的子目录结构，避免不同文件夹里的同名文件互相覆盖。

`-o` 是 `--output` 的简写，下面两条命令效果相同：

```
python md_to_html.py "Markdown编写规范.md" -o ".\output"
python md_to_html.py "Markdown编写规范.md" --output ".\output"
```

`--number-sections`：自动给标题编号

```
python md_to_html.py "Markdown编写规范.md" --toc --number-sections
```

Pandoc 会自动给章节标题编号。如果 Markdown 标题中已经人工写了“一、”“1.”等编号，不建议再使用该参数，否则可能出现重复编号。

`--browser`：手动指定浏览器

一般不需要使用。只有脚本找不到 Chrome 或 Edge 时，才手动填写浏览器路径：

```
python md_to_html.py "Markdown编写规范.md" --pdf --browser "C:\Program
Files\Google\Chrome\Application\chrome.exe"
```

也可以指定 Edge：

```
python md_to_html.py "Markdown编写规范.md" --pdf --browser "C:\Program Files
(x86)\Microsoft\Edge\Application\msedge.exe"
```

`--lang`：设置 HTML 语言

默认是简体中文 `zh-CN`，通常不需要修改：

```
python md_to_html.py "Markdown编写规范.md" --lang zh-CN
```

六、图片应该怎么写

推荐把图片放在 Markdown 旁边的 `images` 文件夹中：

```
md2pdf
├─ md_to_html.py
├─ Markdown编写规范.md
├─ md_to_html使用说明.md
└─ images
    └─ 使用流程.png
```

Markdown 中使用相对路径：

```
![md2pdf 使用流程](images/使用流程.png)
```

脚本会以当前 Markdown 所在目录为起点查找图片，并把本地图片嵌入生成的 HTML。因此生成的 HTML 可以单独复制，不需要再携带 `images` 文件夹。

如果提示“本地图片不存在”，请检查：

- 图片是否真的放在对应目录；
- 文件名和扩展名是否正确；
- Markdown 路径是否相对于当前 `.md` 文件；
- 路径中是否有多余空格。

七、怎么看懂运行结果

正常完成时会看到类似输出：

```
找到 1 个 Markdown 文档，开始转换.....
```

```
完成：Markdown编写规范.html
```

```
PDF 完成：Markdown编写规范.pdf
```

```
处理结束：HTML 成功 1 个，跳过 0 个；PDF 成功 1 个，跳过 0 个；格式警告 0 条；失败 0 个。
```

几个数字的含义：

- `成功`：本次新生成或重新生成的文件数；
- `跳过`：因为使用了 `--skip-existing` 而没有重复生成的文件数；
- `格式警告`：Markdown 预检查发现的问题数量；
- `失败`：没有完成转换的 Markdown 文件数。

脚本结束码为 0 表示全部完成；结束码为 1 表示至少有一个文件失败。批量转换时，一个文件失败不会阻止脚本继续处理后面的文件。

八、常见问题

提示 “未找到 Pandoc”

表示 Pandoc 没有安装，或者脚本找不到 `pandoc.exe`。

先在 PowerShell 中检查：

```
pandoc --version
```

如果仍然找不到，可以安装 Pandoc，或者设置 `PANDOC_PATH` 环境变量指向 `pandoc.exe`。

提示 “未找到 Chrome 或 Edge”

只有使用 `--pdf` 时才需要浏览器。可以确认 Chrome 或 Edge 已安装，或者使用 `--browser` 手动指定浏览器位置。

HTML 正常，但目录出现了一整段正文

通常是 `---` 紧贴上一段文字，Pandoc 把那段文字识别成了标题。

错误写法：

```
> 这是一段提示。  
---
```

正确写法：

```
> 这是一段提示。  
  
---
```

运行时加入 `--strict`，可以在转换前发现这类问题。

中文是否会乱码

HTML 使用 UTF-8，并优先使用微软雅黑等中文字体。PDF 由 Chrome 或 Edge 的打印引擎生成，正常情况下不会出现中文乱码。

如果 Markdown 本身不是 UTF-8，脚本还会尝试按 GB18030 读取。但为了避免不同电脑表现不一致，仍建议把源文件统一保存为 UTF-8。

PDF 没有更新

检查命令中是否使用了 `--skip-existing`。如果用了，该参数会跳过已有 PDF。去掉参数后重新运行即可覆盖更新。

路径中有中文或空格怎么办

把整个路径放进英文双引号中：

```
python md_to_html.py ".\Markdown编写规范.md" --pdf
```

不要删除路径中原本存在的空格，也不要使用中文引号。

九、参数速查表

参数	是否必需	作用
<code>input</code>	否	Markdown 文件或文件夹路径；省略后会提示输入
<code>-o</code> 、 <code>--output</code>	否	指定 HTML 和 PDF 输出位置
<code>--toc</code>	否	生成一至三级标题目录
<code>--pdf</code>	否	在生成 HTML 后继续生成 PDF
<code>--strict</code>	否	有格式警告时停止转换对应文件
<code>--skip-existing</code>	否	分别跳过已有 HTML 和 PDF
<code>--number-sections</code>	否	自动给章节标题编号
<code>--browser</code>	否	手动指定 Chrome 或 Edge 路径
<code>--lang</code>	否	设置 HTML 语言，默认 <code>zh-CN</code>